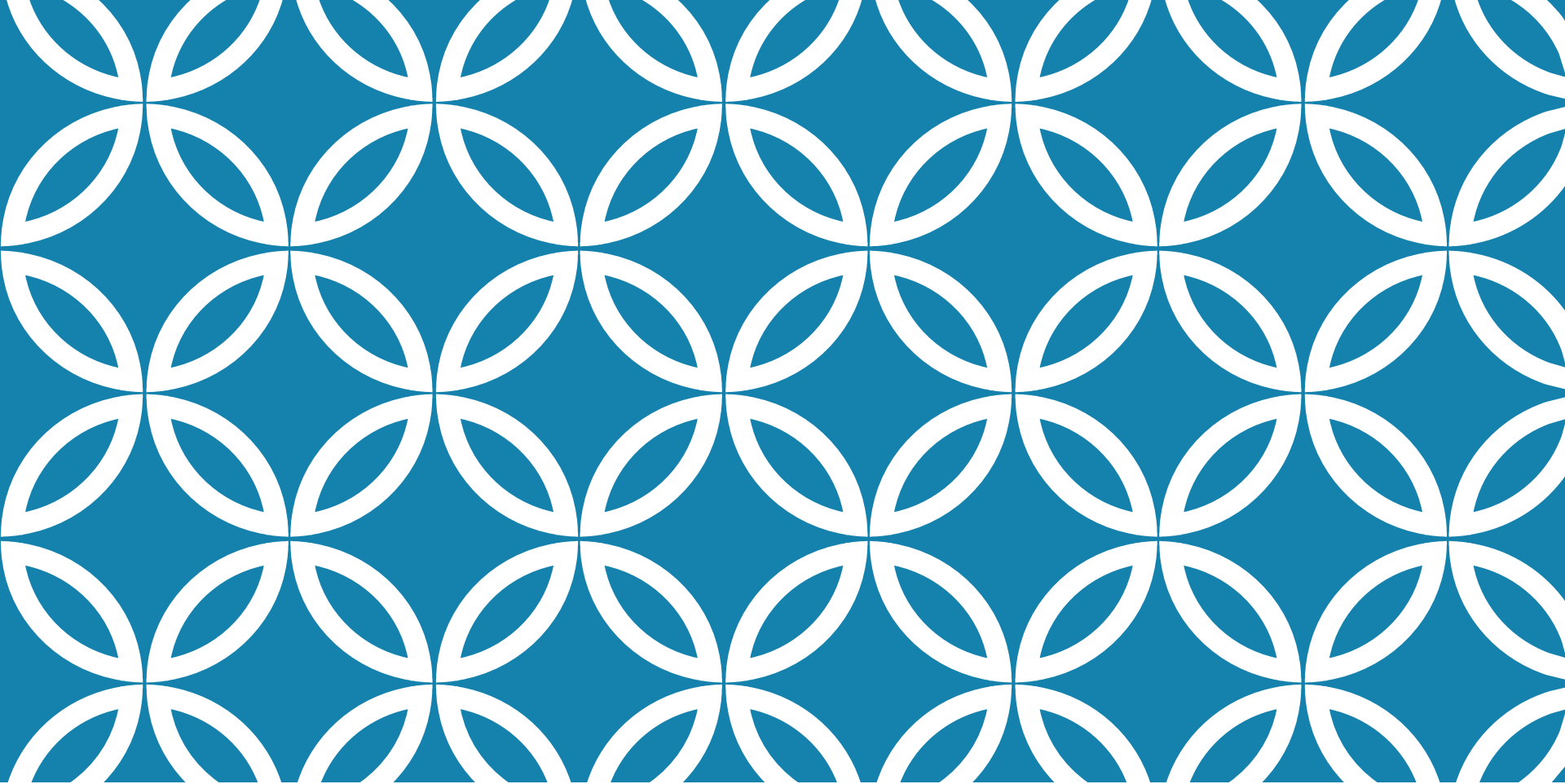




IT SOCIETY

INTRODUCTION

DATABASES

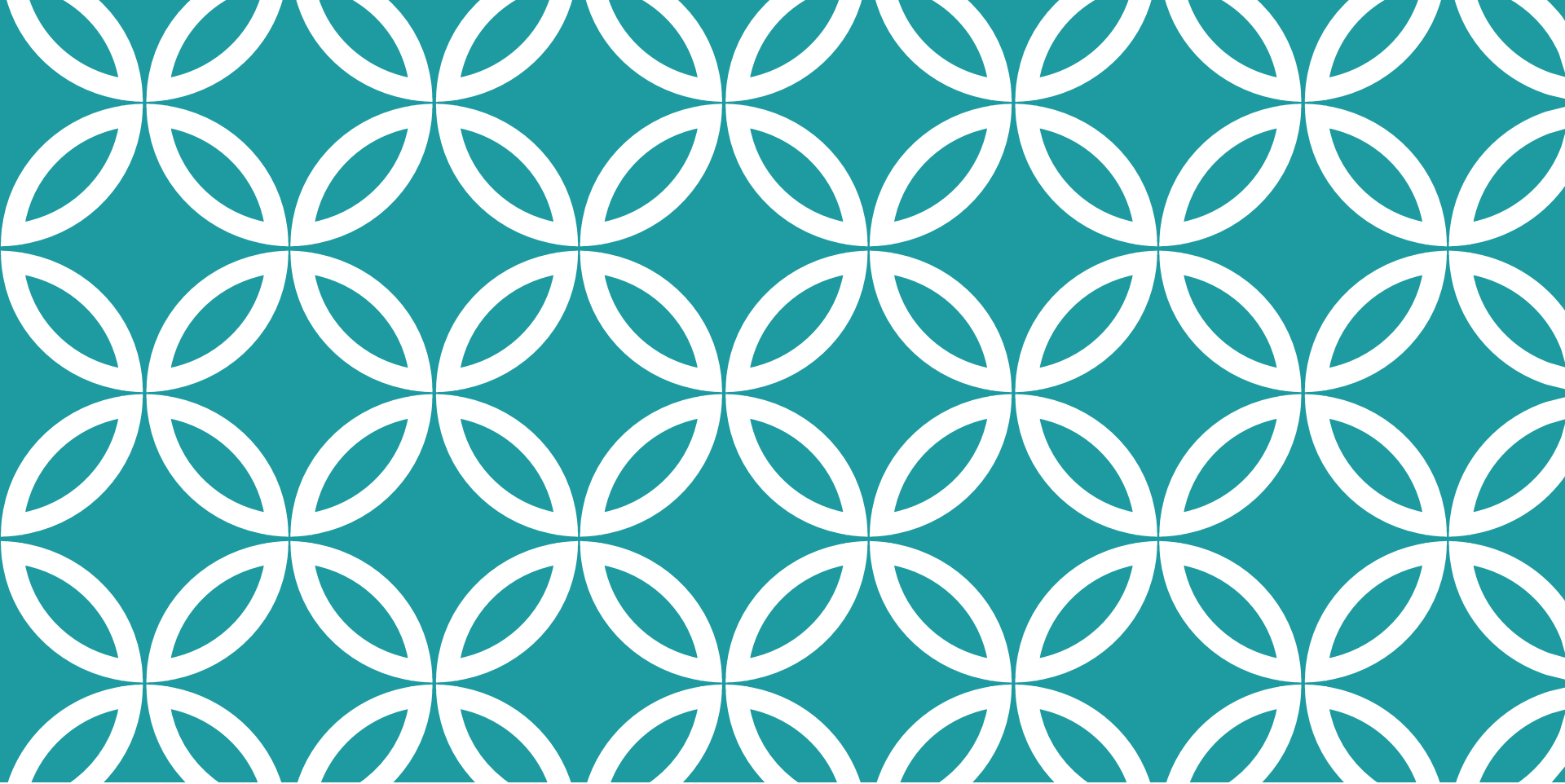
TABLE OF CONTENT

<u>Introduction</u>	5
<u>Constraints</u>	6
<u>PRIMARY KEY</u>	7
<u>AUTO INCREMENT</u>	13
<u>FOREIGN KEY</u>	17
<u>UNIQUE</u>	19
<u>CHECK</u>	21
<u>NOT NULL</u>	23
<u>DEFAULT</u>	25
<u>CONCLUSION</u>	26



RELATIONAL DATABASES





CONSTRAINTS

**PRIMARY KEY,
FOREIGN KEY,
UNIQUE,
CHECK,
NOT NULL &
DEFAULT**

INTRODUCTION

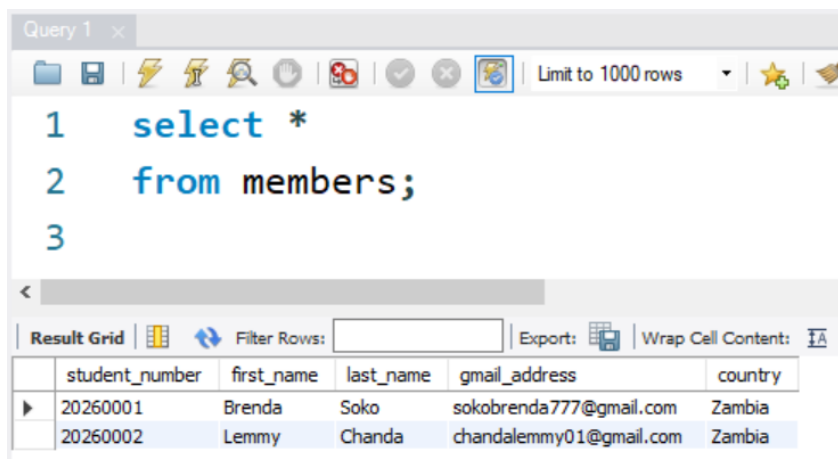
- In MySQL, a **constraint** is a predefined rule applied to a table's columns to restrict the type of data that can be stored in them.
- These are –
 1. PRIMARY KEY
 2. FOREIGN KEY
 3. UNIQUE
 4. NOT NULL
 5. CHECK
 6. DEFAULT

CONSTRAINTS

CONSTRAINT	PURPOSE
PRIMARY KEY	Uniquely identifies each row; no NULL allowed.
FOREIGN KEY	Links a column to a PRIMARY KEY in another table.
UNIQUE	Ensures all values in a column are distinct.
NOT NULL	Prevents NULL values in a column.
CHECK	Validates values against a logical condition.
DEFAULT	Provides a default value when none is supplied.

11. PRIMARY KEY

- When selecting a column where you need to apply the primary key, you need to consider –
 1. The column can not contain NULL values
 2. The column must be UNIQUE
- This leaves us with one column which is the student_number



The screenshot shows a database query editor window titled "Query 1". The query text is:

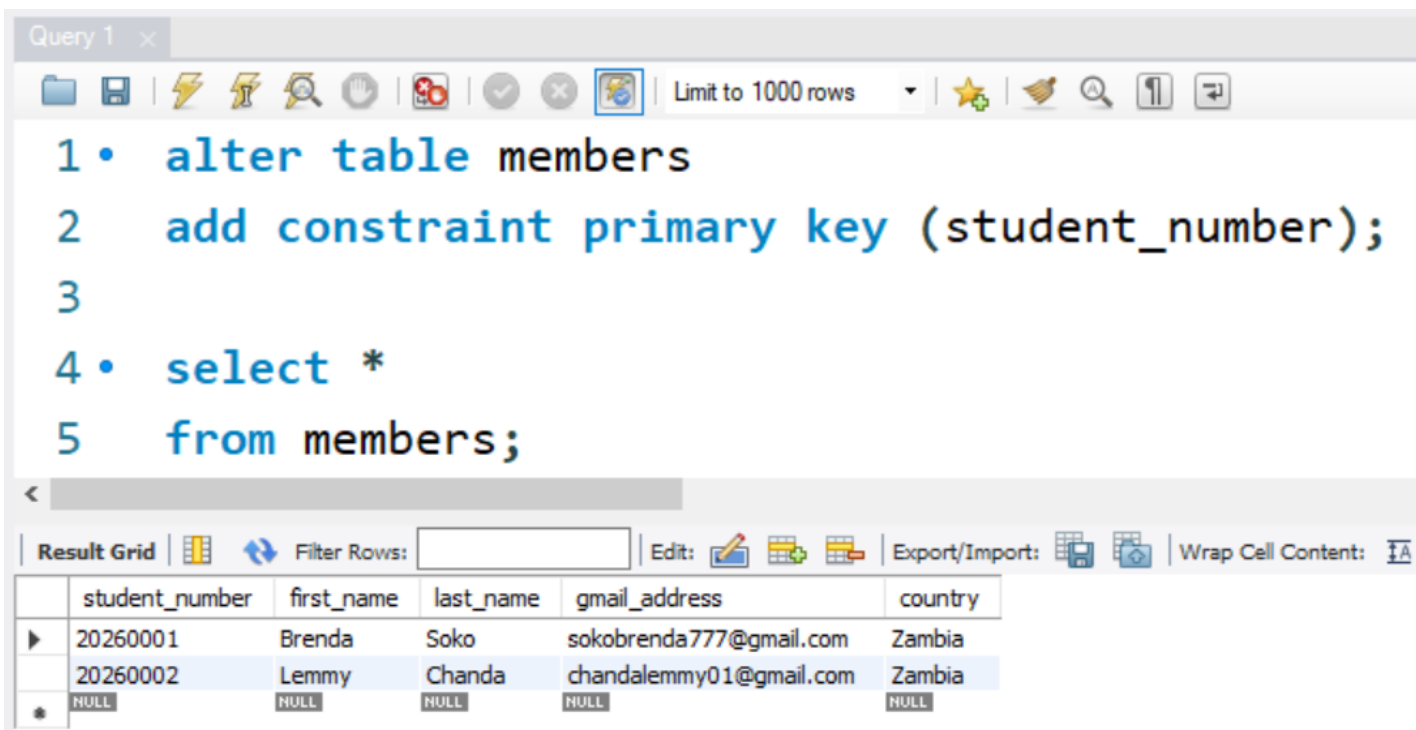
```
1 select *  
2 from members;  
3
```

Below the query editor, there is a "Result Grid" showing the results of the query. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The results are as follows:

student_number	first_name	last_name	gmail_address	country
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260002	Lemmy	Chanda	chandalemm01@gmail.com	Zambia

11. PRIMARY KEY (CONTINUES...)

- Let's apply the primary key on the target field



The screenshot shows a database query editor interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The query editor contains the following SQL commands:

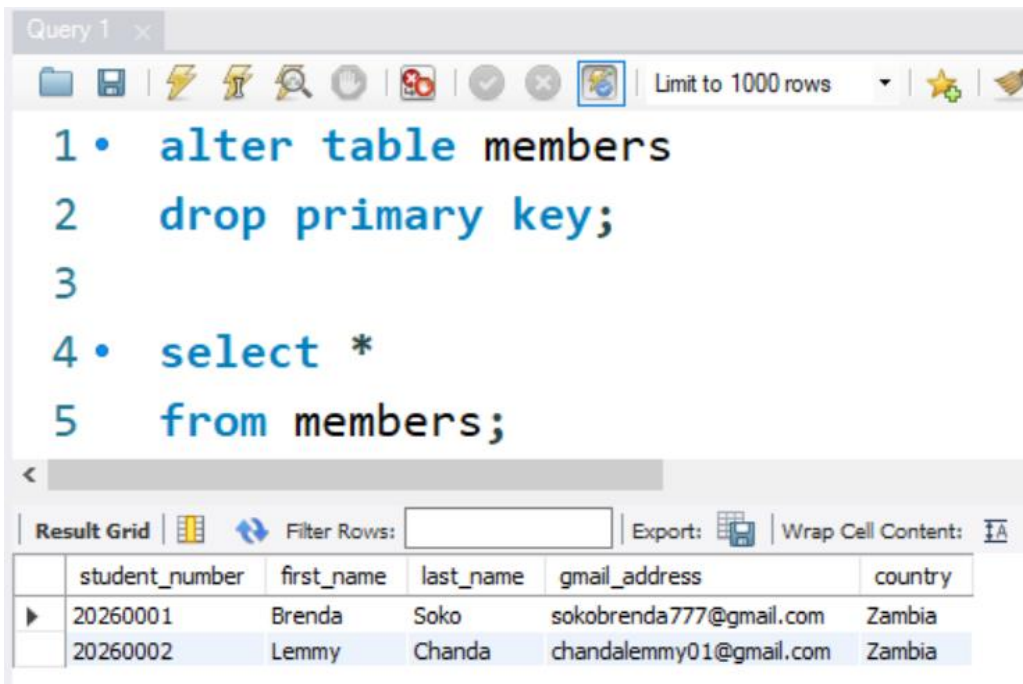
```
1 • alter table members
2   add constraint primary key (student_number);
3
4 • select *
5   from members;
```

Below the query editor is a 'Result Grid' showing the data from the 'members' table. The grid has columns for 'student_number', 'first_name', 'last_name', 'gmail_address', and 'country'. The first two rows contain data for Brenda Soko and Lemmy Chanda. A third row is highlighted with a blue background and contains NULL values for all columns.

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmy01@gmail.com	Zambia
★	NULL	NULL	NULL	NULL	NULL

11. PRIMARY KEY (CONTINUES...)

- To test, the primary key concept, first let's drop it and see how the column was treating data



The screenshot shows a database query editor interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following commands:

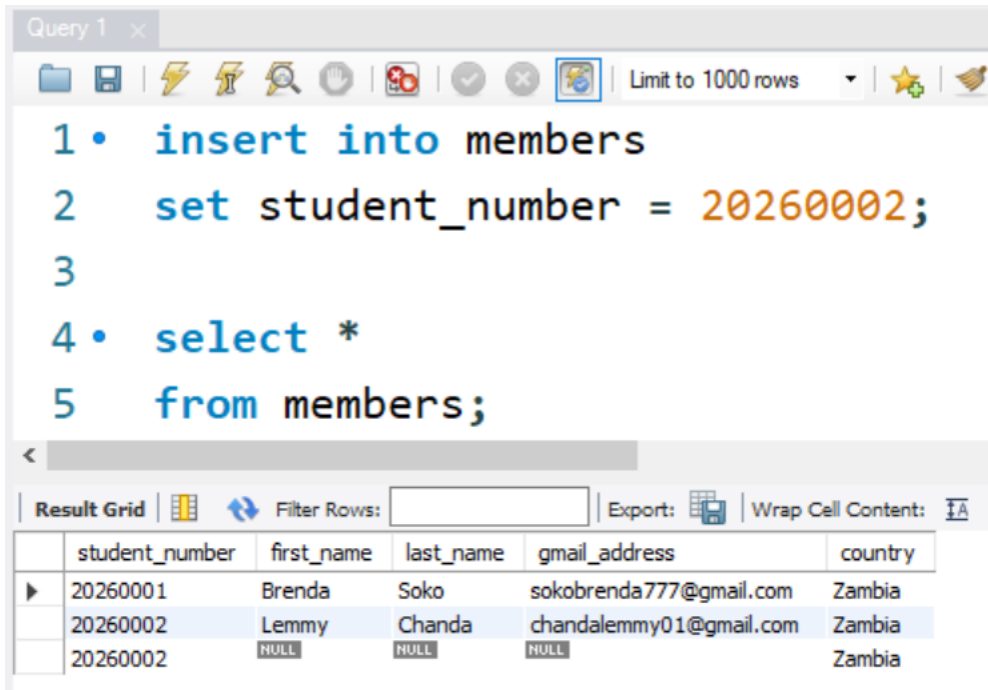
```
1 • alter table members
2   drop primary key;
3
4 • select *
5   from members;
```

Below the editor is the 'Result Grid' section, which displays the data from the 'members' table after the primary key has been dropped. The grid has columns for 'student_number', 'first_name', 'last_name', 'gmail_address', and 'country'.

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia

11. PRIMARY KEY (CONTINUES...)

- Now, will attempt to add a new student with the same number.



The screenshot shows a database query editor with a toolbar and a result grid. The query is as follows:

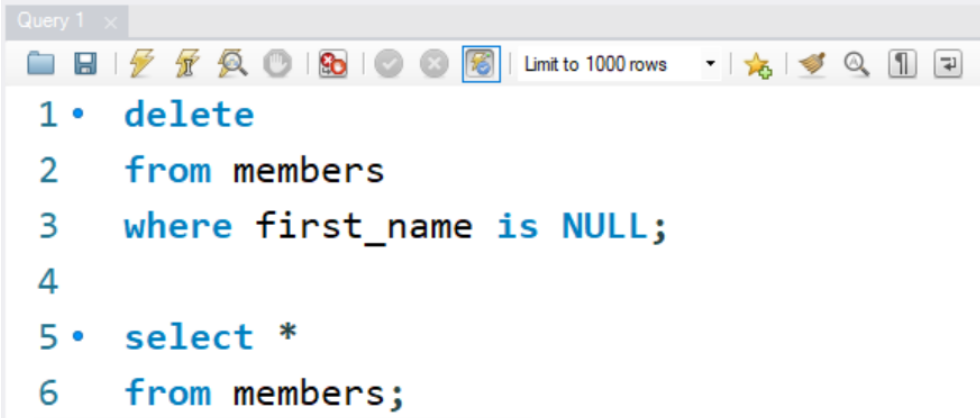
```
1 • insert into members
2   set student_number = 20260002;
3
4 • select *
5   from members;
```

The result grid shows the following data:

student_number	first_name	last_name	gmail_address	country
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
20260002	NULL	NULL	NULL	Zambia

11. PRIMARY KEY (CONTINUES...)

- **NOTE:** The same student_number has been accepted, which is not supposed to be the case in an academic set up.
- For this reason, we need to apply that primary key again, then let's attempt to add duplicate data.



Query 1 x

Limit to 1000 rows

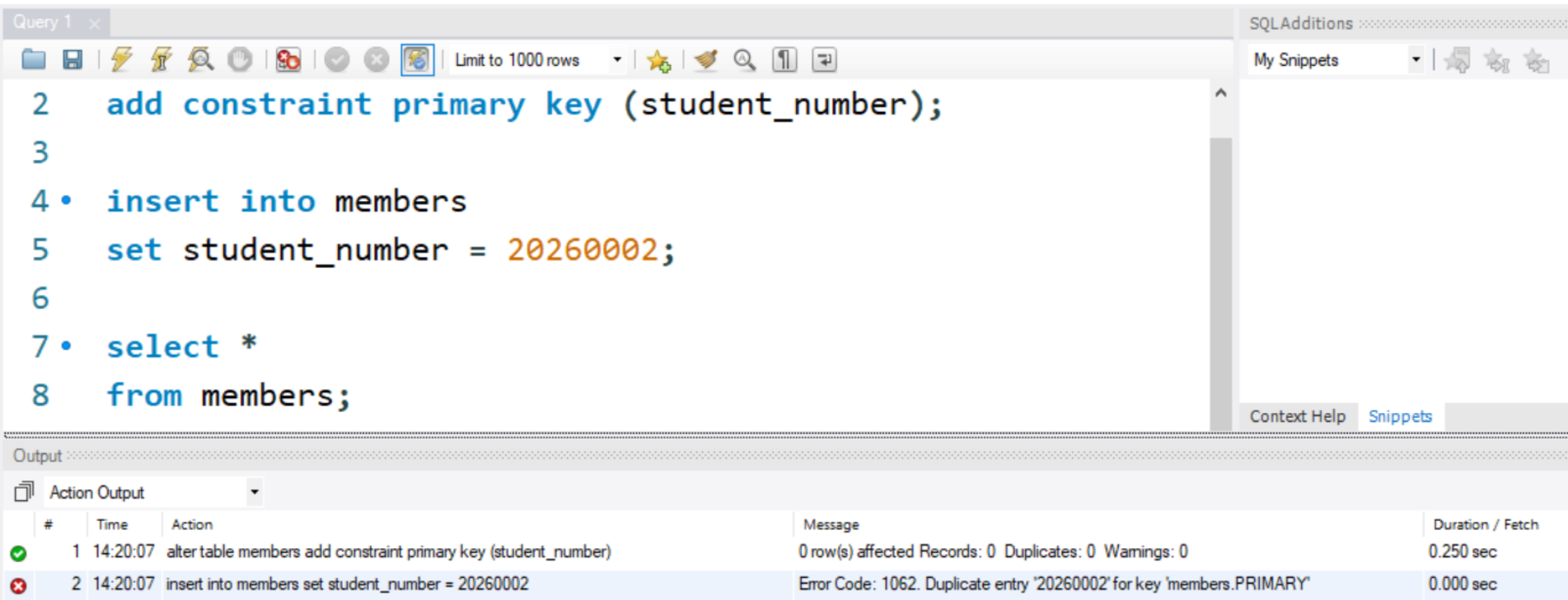
```
1 • delete
2   from members
3   where first_name is NULL;
4
5 • select *
6   from members;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia

11. PRIMARY KEY (CONTINUES...)

- Now notice the error message



The screenshot shows a database management interface with a query editor and an output window. The query editor contains the following SQL code:

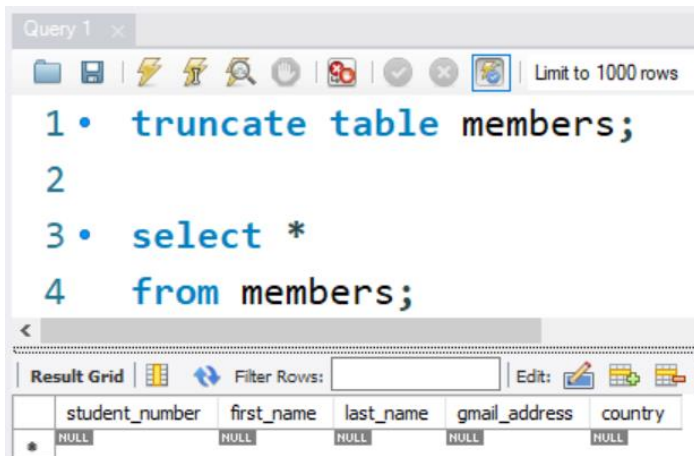
```
2  add constraint primary key (student_number);
3
4 • insert into members
5  set student_number = 20260002;
6
7 • select *
8  from members;
```

The output window shows the results of the queries. The first query (alter table members add constraint primary key (student_number)) was successful. The second query (insert into members set student_number = 20260002) failed with an error message:

#	Time	Action	Message	Duration / Fetch
1	14:20:07	alter table members add constraint primary key (student_number)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.250 sec
2	14:20:07	insert into members set student_number = 20260002	Error Code: 1062. Duplicate entry '20260002' for key 'members.PRIMARY'	0.000 sec

11.1 AUTO_INCREMENT

- An **auto_increment** field is a numeric column that automatically generates a unique number, when a new record is inserted into a table.
- For this reason, it is applied on a numeric column with a PRIMARY KEY.
- By default, auto_increment starts at 1.



The screenshot shows a database query editor window titled "Query 1". The SQL code entered is:

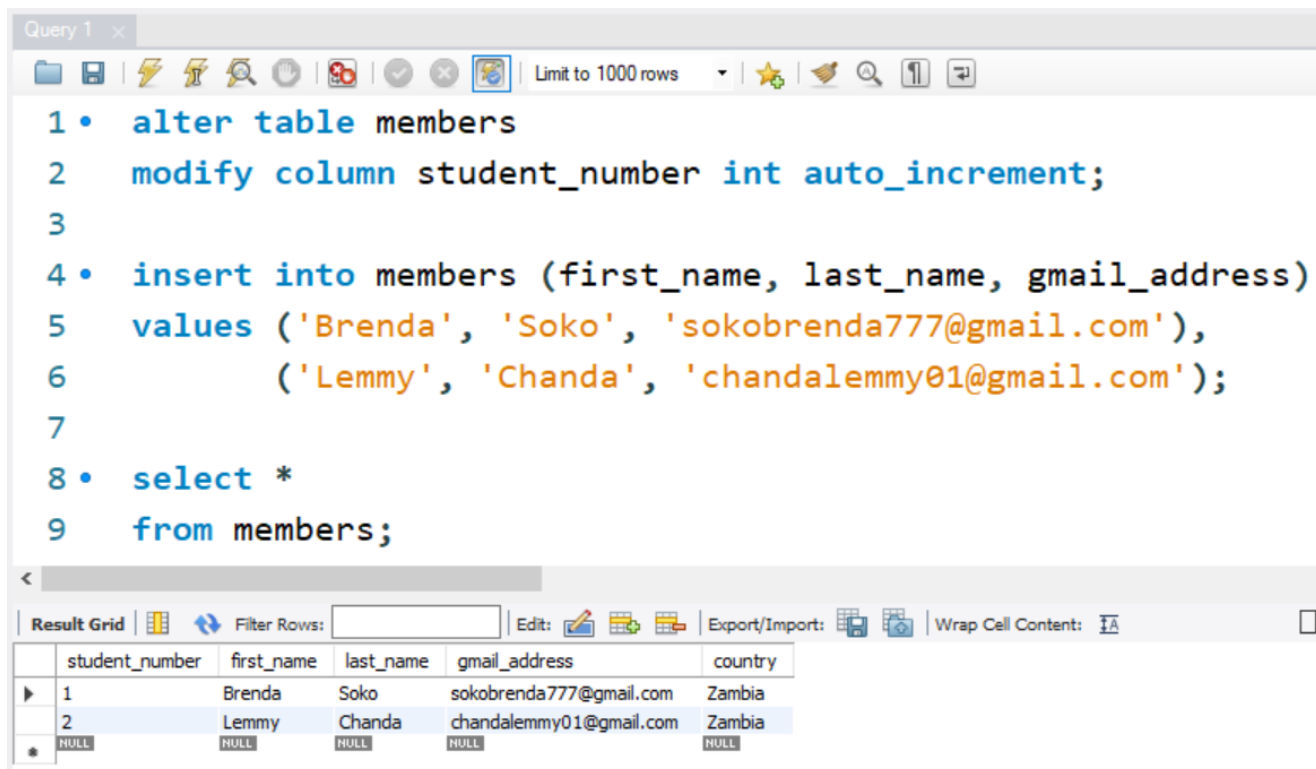
```
1 • truncate table members;
2
3 • select *
4   from members;
```

Below the code editor is a "Result Grid" showing the results of the query. The grid has five columns: student_number, first_name, last_name, gmail_address, and country. The first row shows all columns with NULL values.

	student_number	first_name	last_name	gmail_address	country
*	NULL	NULL	NULL	NULL	NULL

11.1 AUTO_INCREMENT (CONTINUES...)

- Let's add the auto_increment and insert some records.



The screenshot shows a database query editor window titled "Query 1". The SQL code entered is as follows:

```
1 • alter table members
2   modify column student_number int auto_increment;
3
4 • insert into members (first_name, last_name, gmail_address)
5   values ('Brenda', 'Soko', 'sokobrenda777@gmail.com'),
6          ('Lemmy', 'Chanda', 'chandalemmmy01@gmail.com');
7
8 • select *
9   from members;
```

Below the code editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The data is as follows:

	student_number	first_name	last_name	gmail_address	country
▶	1	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	2	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

11.1 AUTO_INCREMENT (CONTINUES...)

- We can also set a different starting point for our auto_increment.

The screenshot shows a database query editor window titled "Query 1". The SQL code entered is as follows:

```
1 • alter table members
2   auto_increment = 100;
3
4 • insert into members (first_name, last_name, gmail_address)
5   values ('James', 'Soko', 's0k0j4m3s@gmail.com');
6
7 • select *
8   from members;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The results are as follows:

	student_number	first_name	last_name	gmail_address	country
▶	1	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	2	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
	100	James	Soko	s0k0j4m3s@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

11.1 AUTO_INCREMENT (CONTINUES...)

- This patten is not unique, let's get rid of our the records and start a unique patten.

The screenshot shows a database query editor window titled "Query 1". The SQL code is as follows:

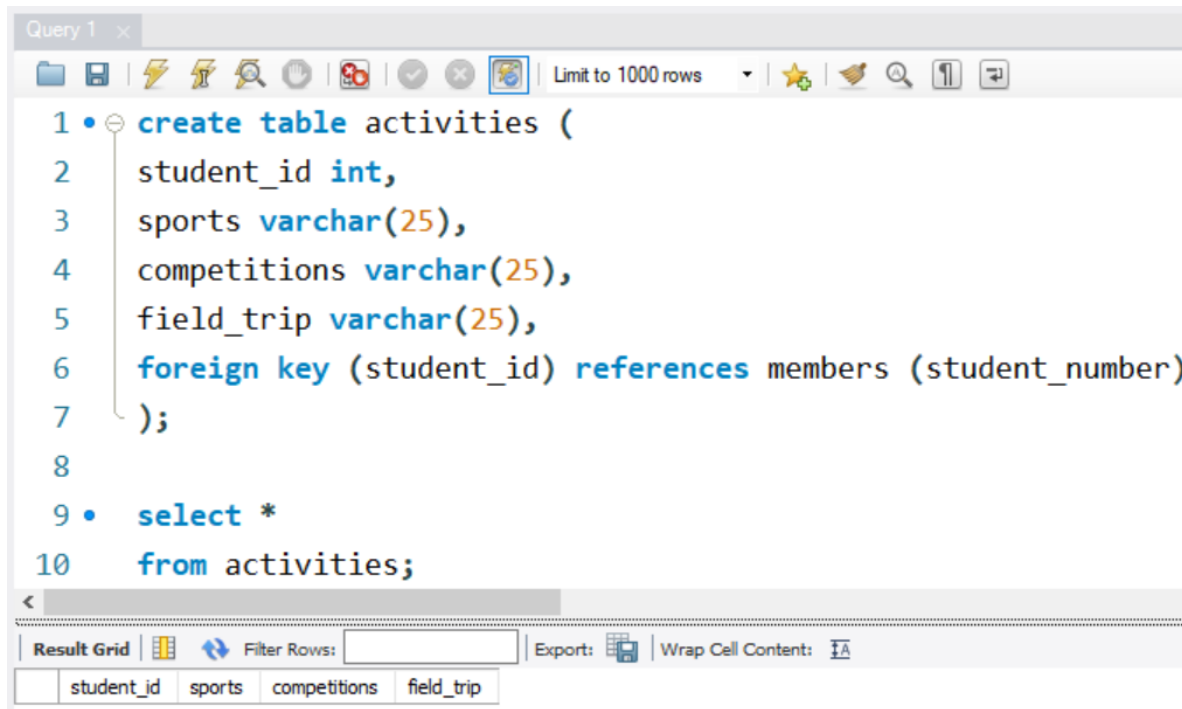
```
1 • delete
2   from members;
3
4 • insert into members (student_number, first_name, last_name, gmail_address)
5   values (20260001, 'Brenda', 'Soko', 'sokobrenda777@gmail.com');
6
7 • insert into members (first_name, last_name, gmail_address)
8   values ('Lemmy', 'Chanda', 'chandalemmmy01@gmail.com');
9
10 • select *
11   from members;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns: student_number, first_name, last_name, gmail_address, and country. The data is as follows:

student_number	first_name	last_name	gmail_address	country
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
NULL	NULL	NULL	NULL	NULL

12. FOREIGN KEY

- For the sake of understanding a foreign key, let's create another table.



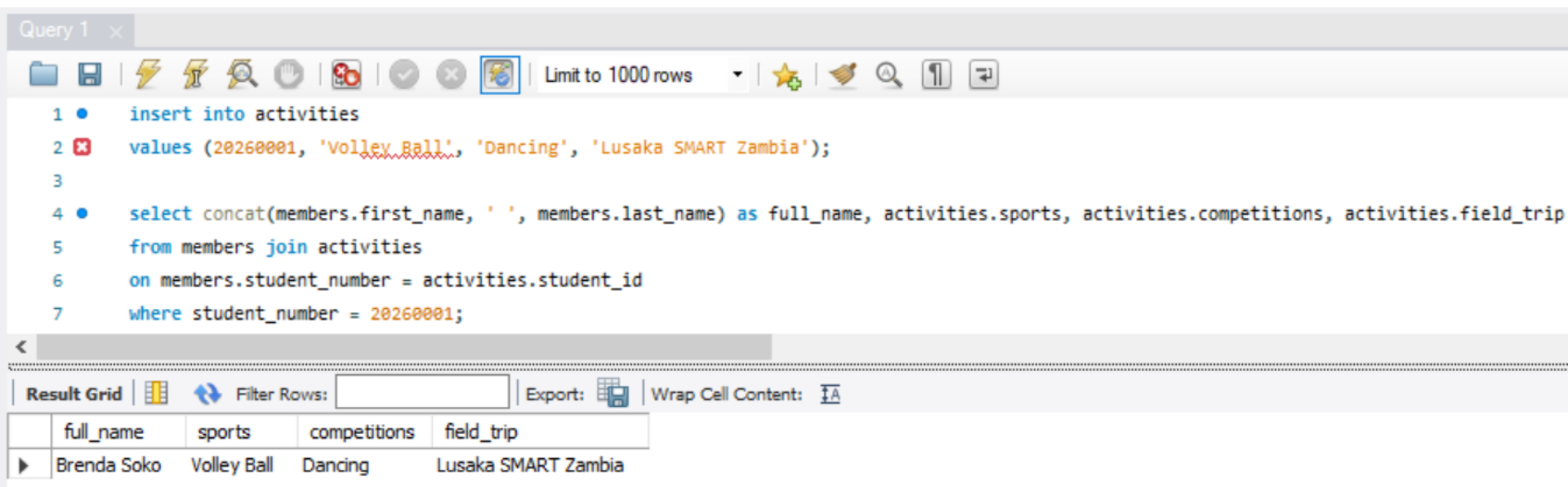
```
Query 1 x
Limit to 1000 rows
1 • create table activities (
2   student_id int,
3   sports varchar(25),
4   competitions varchar(25),
5   field_trip varchar(25),
6   foreign key (student_id) references members (student_number)
7 );
8
9 • select *
10 from activities;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

student_id	sports	competitions	field_trip
------------	--------	--------------	------------

12. FOREIGN KEY (CONTINUES...)

- Related data from different tables can be linked by a foreign key to its primary key from another table.
- Let's take an example below which will be covered in the next topic.



The screenshot shows a database query editor interface. At the top, there's a tab labeled "Query 1". Below the tab is a toolbar with various icons for file operations, execution, and viewing. A dropdown menu shows "Limit to 1000 rows". The main area contains an SQL query:

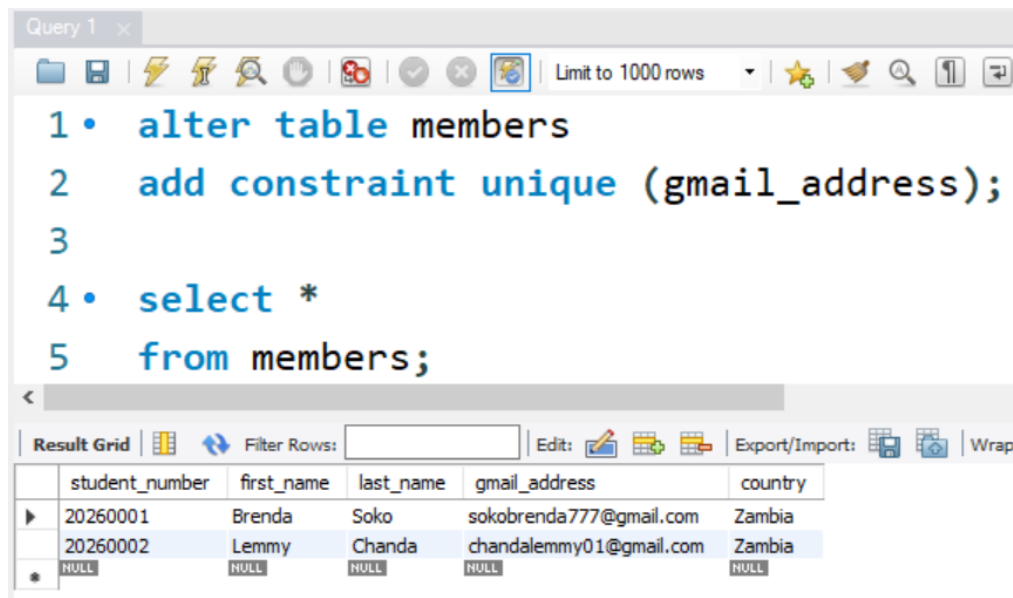
```
1 • insert into activities
2 ✖ values (20260001, 'Volley Ball', 'Dancing', 'Lusaka SMART Zambia');
3
4 • select concat(members.first_name, ' ', members.last_name) as full_name, activities.sports, activities.competitions, activities.field_trip
5 from members join activities
6 on members.student_number = activities.student_id
7 where student_number = 20260001;
```

Below the query, there's a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The results are displayed in a table:

full_name	sports	competitions	field_trip
Brenda Soko	Volley Ball	Dancing	Lusaka SMART Zambia

13. UNIQUE

- Similar to a primary key, again we need to identify which column can have distinct records.
- Our best option for this can be the gmail_address field because email addresses can not be the same.



The screenshot shows a database query editor window titled "Query 1". The SQL code entered is:

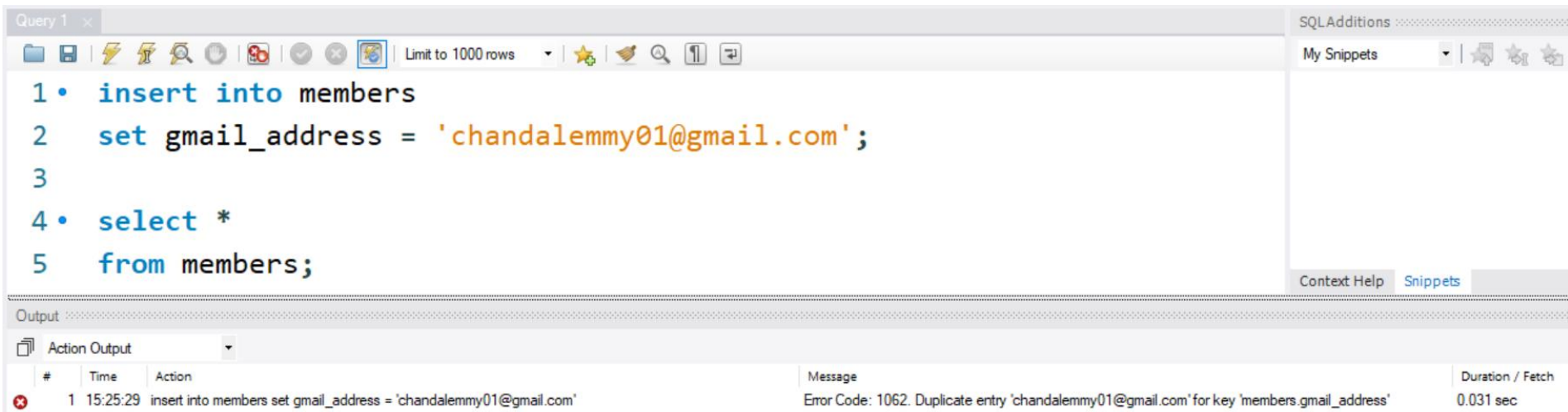
```
1 • alter table members
2   add constraint unique (gmail_address);
3
4 • select *
5   from members;
```

Below the code is a "Result Grid" showing the output of the query. The grid has columns: student_number, first_name, last_name, gmail_address, and country. The data is as follows:

student_number	first_name	last_name	gmail_address	country
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
NULL	NULL	NULL	NULL	NULL

13. UNIQUE (CONTINUES...)

- At this point, we can not have the same email address in that field.



The screenshot shows a database query editor interface. The main area contains the following SQL code:

```
1 • insert into members
2   set gmail_address = 'chandalemmmy01@gmail.com';
3
4 • select *
5   from members;
```

Below the code editor is the 'Output' section, which displays an error message:

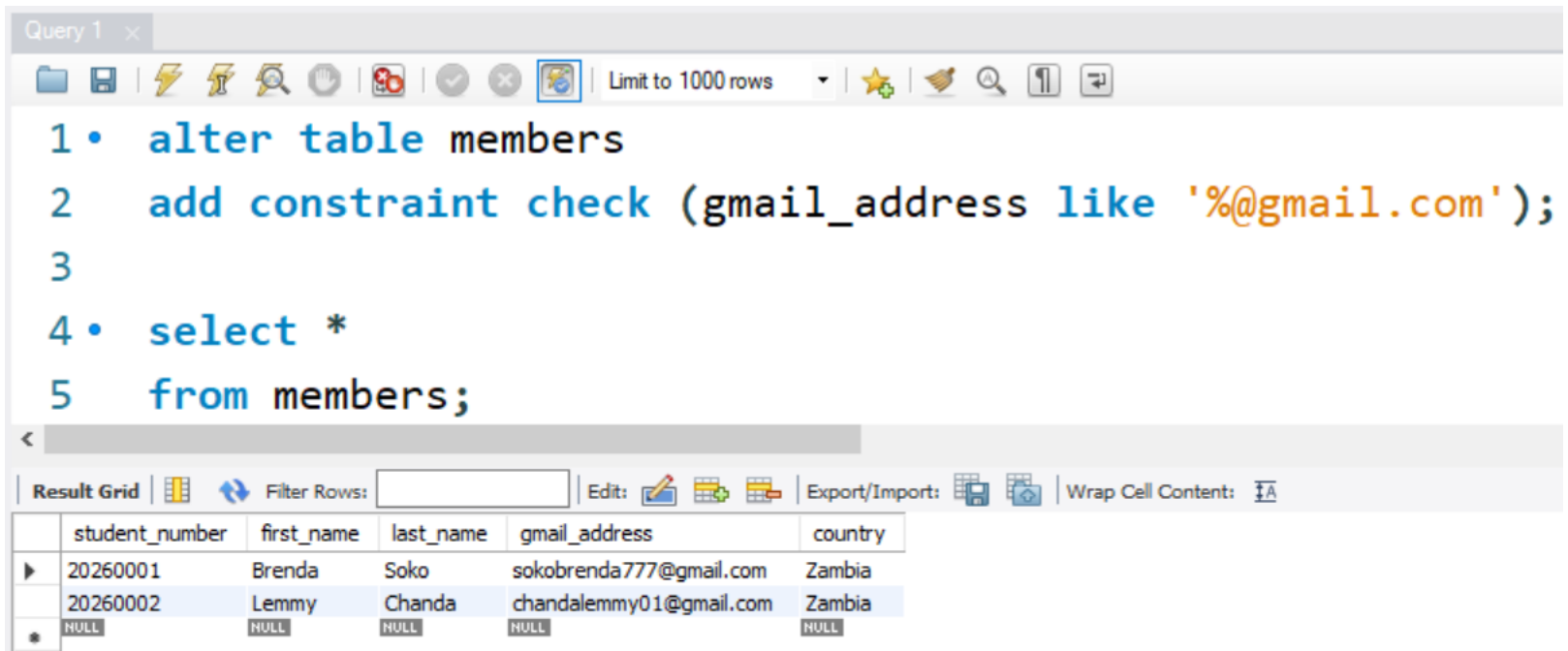
#	Time	Action	Message	Duration / Fetch
1	15:25:29	insert into members set gmail_address = 'chandalemmmy01@gmail.com'	Error Code: 1062. Duplicate entry 'chandalemmmy01@gmail.com' for key 'members.gmail_address'	0.031 sec

The error message indicates a duplicate entry for the email address 'chandalemmmy01@gmail.com' in the 'members' table.

- **NOTE:** At some point, we need to **check** if it's a varied google account.

14. CHECK

- Now, let's make sure we only accept varied email addresses under the google domain.



The screenshot shows a database query editor interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following commands:

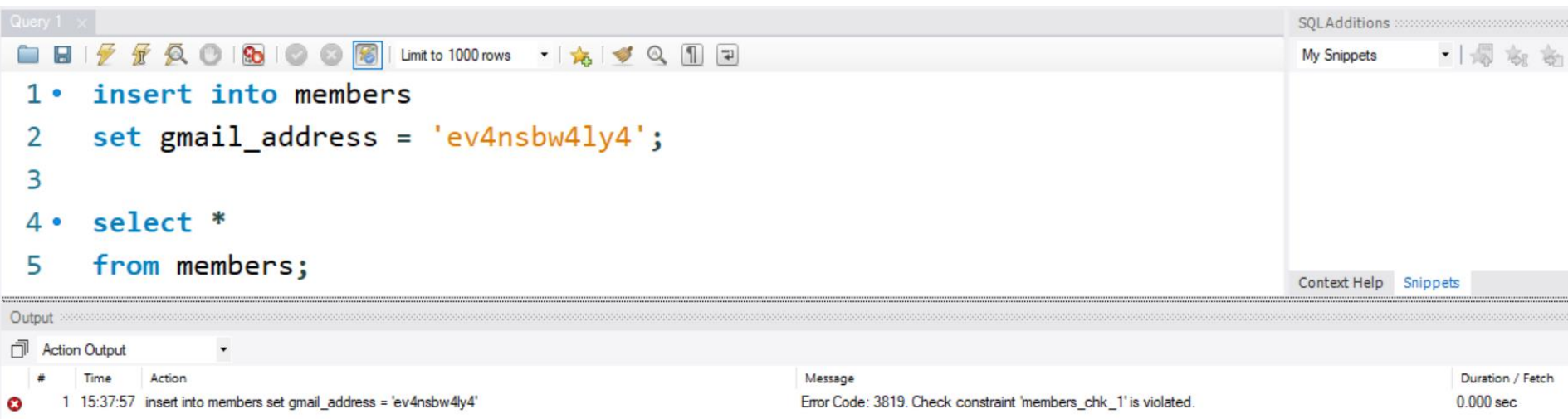
```
1 • alter table members
2   add constraint check (gmail_address like '%@gmail.com');
3
4 • select *
5   from members;
```

Below the editor is a 'Result Grid' showing the data from the 'members' table. The grid has columns for student_number, first_name, last_name, gmail_address, and country. The first two rows contain data for Brenda Soko and Lemmy Chanda, both with gmail addresses. The third row shows NULL values for all columns.

	student_number	first_name	last_name	gmail_address	country
▶	20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
	20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
*	NULL	NULL	NULL	NULL	NULL

14. CHECK (CONTINUES...)

- Will discuss the LIKE operator as we progress with our topics, but for now, just know that % is a pattern that accepts multiple values.
- So %@gmail.com mean, you can type anything but must end with @gmail.com



The screenshot shows a SQL IDE interface. The main editor displays a query with five lines:

```
1 • insert into members
2   set gmail_address = 'ev4nsbw4ly4';
3
4 • select *
5   from members;
```

The right sidebar shows 'SQLAdditions' with 'My Snippets' and 'Context Help' buttons.

The bottom 'Output' pane shows the 'Action Output' tab with a table of execution results:

#	Time	Action
1	15:37:57	insert into members set gmail_address = 'ev4nsbw4ly4'

Below the table, a message box displays the error:

Message
Error Code: 3819. Check constraint 'members_chk_1' is violated.

Duration / Fetch
0.000 sec

15. NOT NULL

- This constraint is straightforward, just identify the field that must not contain NULL values and apply the constraint.
- Suppose we want the gmail_address field not to contain null values.

The screenshot shows a database query editor window titled "Query 1". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL editor contains the following commands:

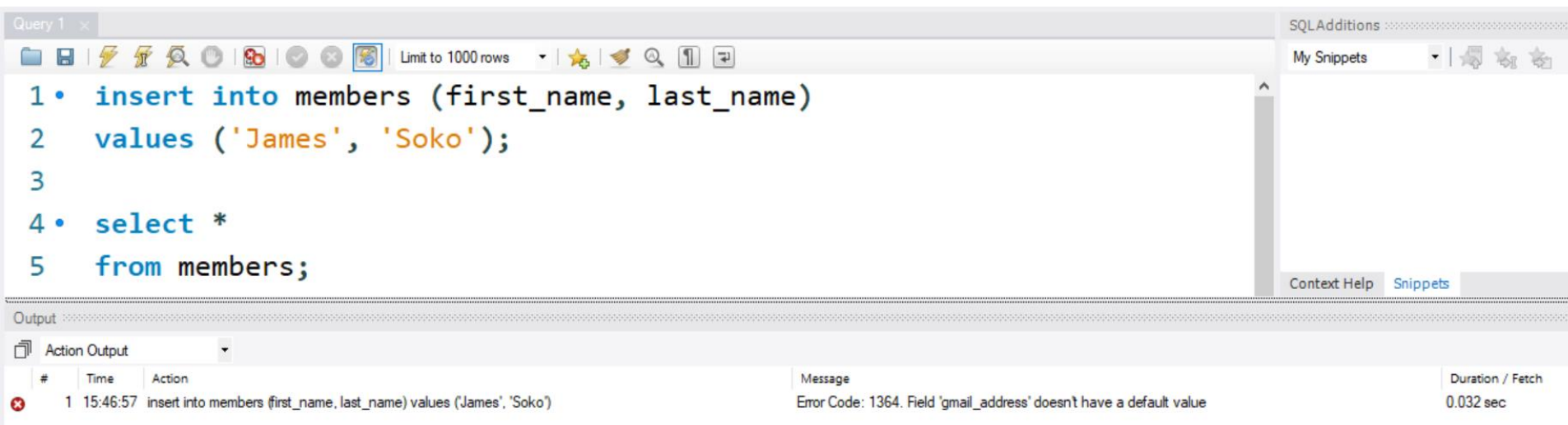
```
1 • alter table members
2   modify column gmail_address varchar(25) not null;
3
4 • select *
5   from members;
```

Below the editor is a "Result Grid" showing the results of the query. The grid has columns: student_number, first_name, last_name, gmail_address, and country. The data is as follows:

student_number	first_name	last_name	gmail_address	country
20260001	Brenda	Soko	sokobrenda777@gmail.com	Zambia
20260002	Lemmy	Chanda	chandalemmmy01@gmail.com	Zambia
NULL	NULL	NULL	NULL	NULL

15. NOT NULL (CONTINUES...)

- Let's put this constraint to the test.



The screenshot shows a SQL client interface with a query editor and an output pane. The query editor contains the following SQL code:

```
1 • insert into members (first_name, last_name)
2   values ('James', 'Soko');
3
4 • select *
5   from members;
```

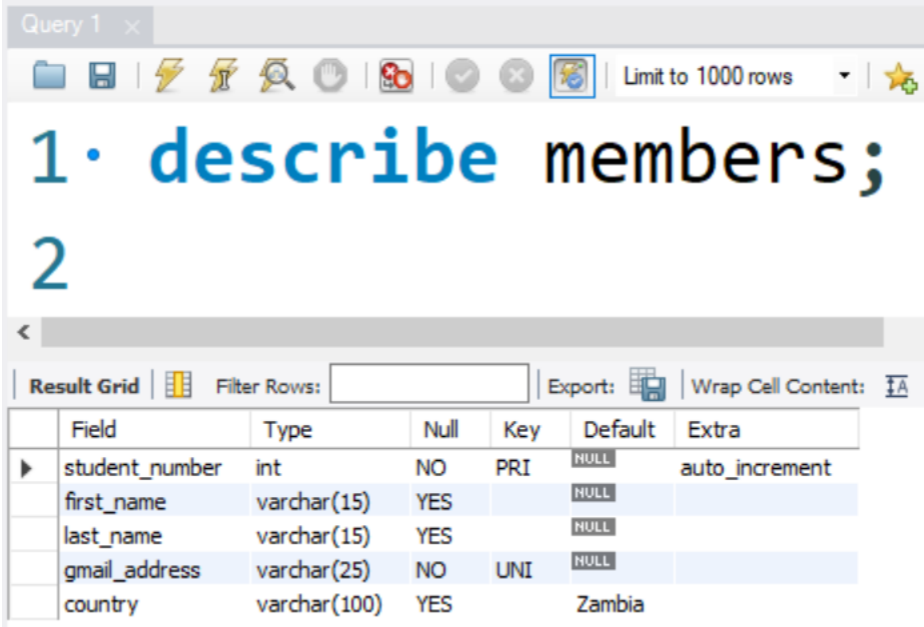
The output pane shows an error message:

#	Time	Action	Message	Duration / Fetch
1	15:46:57	insert into members (first_name, last_name) values ('James', 'Soko')	Error Code: 1364. Field 'gmail_address' doesn't have a default value	0.032 sec

The error message indicates that the field 'gmail_address' does not have a default value, which is causing the insert operation to fail.

16. DEFAULT

- You can tell from our country field that the default value is Zambia.
- Refer to slide number 30 in the first handout how to use the default constraint.
- Let's just check our table description.



The screenshot shows a database query tool interface. At the top, there's a tab labeled "Query 1". Below the tab is a toolbar with various icons. The main area displays a SQL query: `1. describe members;` and `2`. Below the query, there's a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid itself is a table with the following data:

Field	Type	Null	Key	Default	Extra
student_number	int	NO	PRI	NULL	auto_increment
first_name	varchar(15)	YES		NULL	
last_name	varchar(15)	YES		NULL	
gmail_address	varchar(25)	NO	UNI	NULL	
country	varchar(100)	YES		Zambia	

CONCLUSION

- At this point, we now have an understanding about **MySQL constraints**.
- In this third handout, our focus was to understand the use of a **PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL** and **DEFAULT** constraint.



UP NEXT... JOINS